# AUTOSAR COM Stack

# Agenda

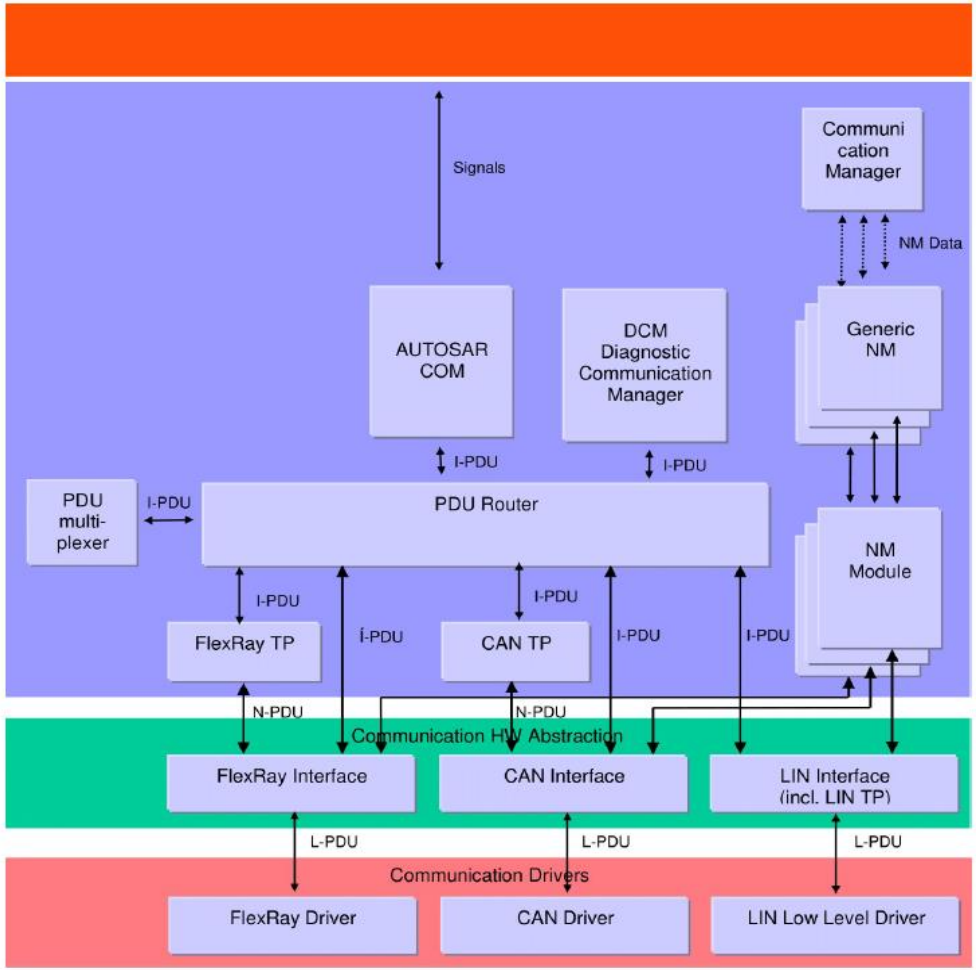
- Introduction
- CAN
- CAN Driver Module
- CAN IF Module
- CanTP Module
- PduR Module
- COM Module

# CAN TP Module

- CanTp is the module between the PDU Router and the CAN Interface module
- The main purpose of the CAN TP module is to segment and reassemble CAN I-PDUs longer than 8 bytes or longer than 64 bytes in case of CAN FD.
- CanTp only handles transport protocol frames (i.e. SF, FF, CF and FC PDUs), depending on the N-PDU ID, the CAN Interface has to forward an I-PDU to CanTp or PduR.

***CanTp Functionalities:***

- Segmentation of data in transmit direction
- Reassembling of data in receive direction
- Control of data flow
- Detection of errors in segmentation sessions
- Transmit cancellation
- Receive cancellation


Signals
Communi cation Manager
NM Data
AUTOSAR COM
DCM Diagnostic Communication Manager
Generic NM
I-PDU
I-PDU
PDU multi- plexer
I-PDU
PDU Router
NM Module
I-PDU
I-PDU
FlexRay TP
Í-PDU
CAN TP
I-PDU
I-PDU
N-PDU
N-PDU
Communication HW Abstraction
FlexRay Interface
CAN Interface
LIN Interface (incl. LIN TP)
L-PDU
L-PDU
L-PDU
Communication Drivers
FlexRay Driver
CAN Driver
LIN Low Level Driver

# CAN Transport Layer interactions

The CanTp's upper interface offers the PduR module global access, to transmit and receive data. This access is achieved by CAN N-SDU identifier (CAN NSduId). CAN NSduId refers to a constant data structure which consists of attributes describing CAN N-SDU. Each CAN N-SDU specific data structure may contain attributes such as: type of N-SDU (Tx or Rx), its addressing format, L-SDU identifier of this message or other attributes that are useful for implementation.

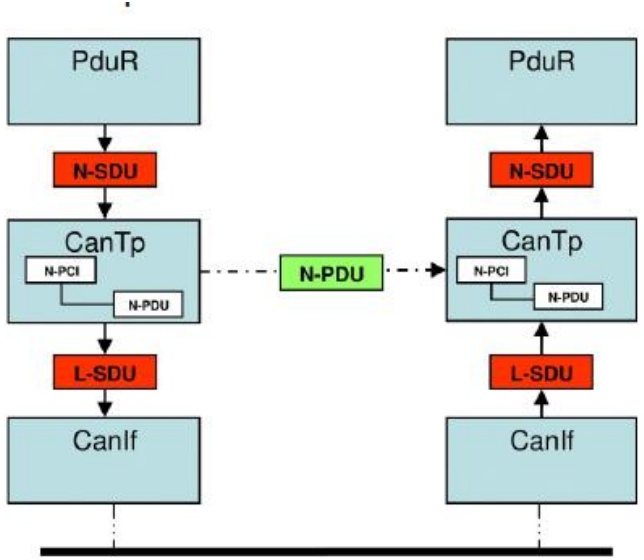



Figure 5.1: CAN Transport Layer interactions

## Services provided to upper layer

- Initialization and shutdown
- Communication services

## Initialization and shutdown

- The CanTp module shall have two internal states, CANTP_OFF and CANTP_ON.
- The CanTp module shall change to the internal state CANTP_ON when the CanTp has been successfully initialized with CanTp_Init().
- The CanTp module shall perform segmentation and reassembly tasks only when the CanTp is in the CANTP_ON state
- The function CanTp_Init shall initialize all global variables of the module and sets all transport protocol connections
- The function CanTp_Shutdown shall stop the CanTp module properly

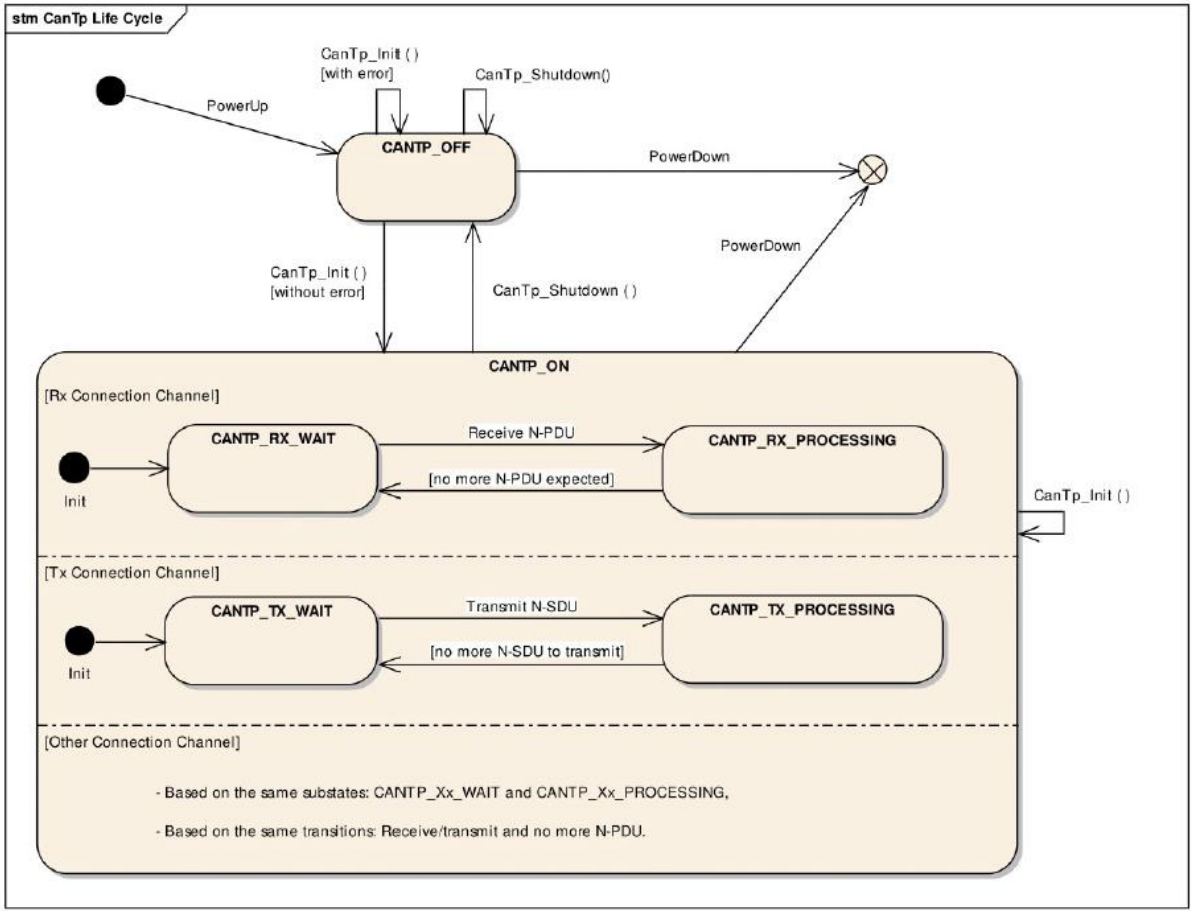



**Figure 7.1: CAN Transport Layer life cycle**

## Transmit request

- The transmit operation, **CanTp_Transmit**(), will allow upper layers to ask for datatransfer using CAN transport protocol facilities
- The function CanTp_Transmit() shall be asynchronous.
- After the transmit request was accepted, the CanTp module shall notify its upper layer if the N-SDU transfer is fully processed (successfully or not)

## Transmit cancellation

- The transmit cancellation feature allows the upper layer to cancel a transmission in progress.
- **Use case: Cancel a diagnostic transmission due to the reception of another diagnostic** protocol with higher priority.
- **Api -> CanTp_CancelTransmit()**

# Services provided to the lower layer

## Transmit confirmation

- The CanIf module shall call the transmit confirmation function to notify the CAN Transport Layer that a CAN frame transmission, requested by the CanTp, has been performed successfully or not.
- **API -> CanTp_TxConfirmation().**

## Reception indication

- The CanIf module shall call the reception indication function to notify the CanTp module that a new CAN N-PDU frame (i.e. a transport protocol frame) has been received.
- The reception indication can be performed in ISR context according to CanIf configuration.
- **API -> CanTp_RxIndication().**

# N-SDU Reception

- When receiving an SF or an FF N-PDU, the CanTp module shall notify the upper layer (PDU Router) about this reception using the **PduR_CanTpStartOfReception** function.
- The upper layer will **reserve and lock a buffer** for reception of the N-SDU.
- At the reception of a FF or last CF of a block, the CanTp module shall **start a time-out N_Br** before calling PduR_CanTpStartOfReception or PduR_CanTpCopyRxData.
- After reception of each Consecutive Frame the CanTp module shall call the **PduR_CanTpCopyRxData**() function with a PduInfo pointer containing data buffer and data length
- Once all the data is copied then CanTp will call **PduR_CanTpRxIndication**()

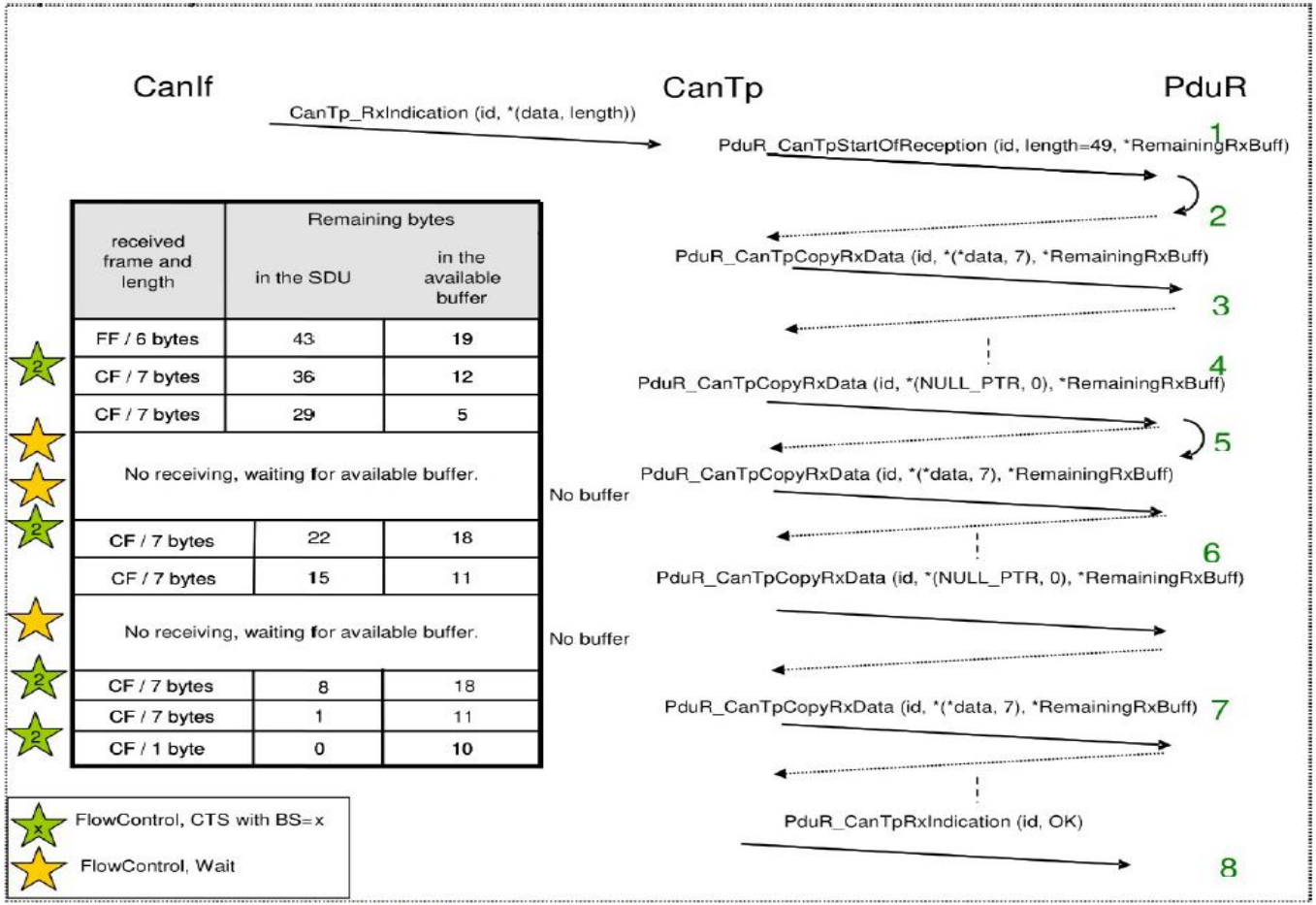



**Figure 7.5: Example of receiving process**

# N-SDU Transmission

- The upper layer asks for the transmission of a N-SDU by calling **CanTp_Transmit**()
- The parameters of **CanTp_Transmit**() describe the CAN **NSduId** and the full **Tx N-SDU length to be sent** .
- After a transmission request from upper layer, the CanTp module **shall start time-out N_Cs** before the call of **PduR_CanTpCopyTxData()**. If no data is available before the timer elapsed, the CanTp module shall abort the communication.
- The CanTp module shall call the **PduR_CanTpCopyTxData** service **for each segment that is sent (SF, FF and CF)**. The upper layer copy the transmit data on the PduInfoType structure.
- When the transport transmission session is successfully completed, the CanTp module shall call a notification service of the upper layer, **PduR_CanTpTxConfirmation**(), with the result E_OK.

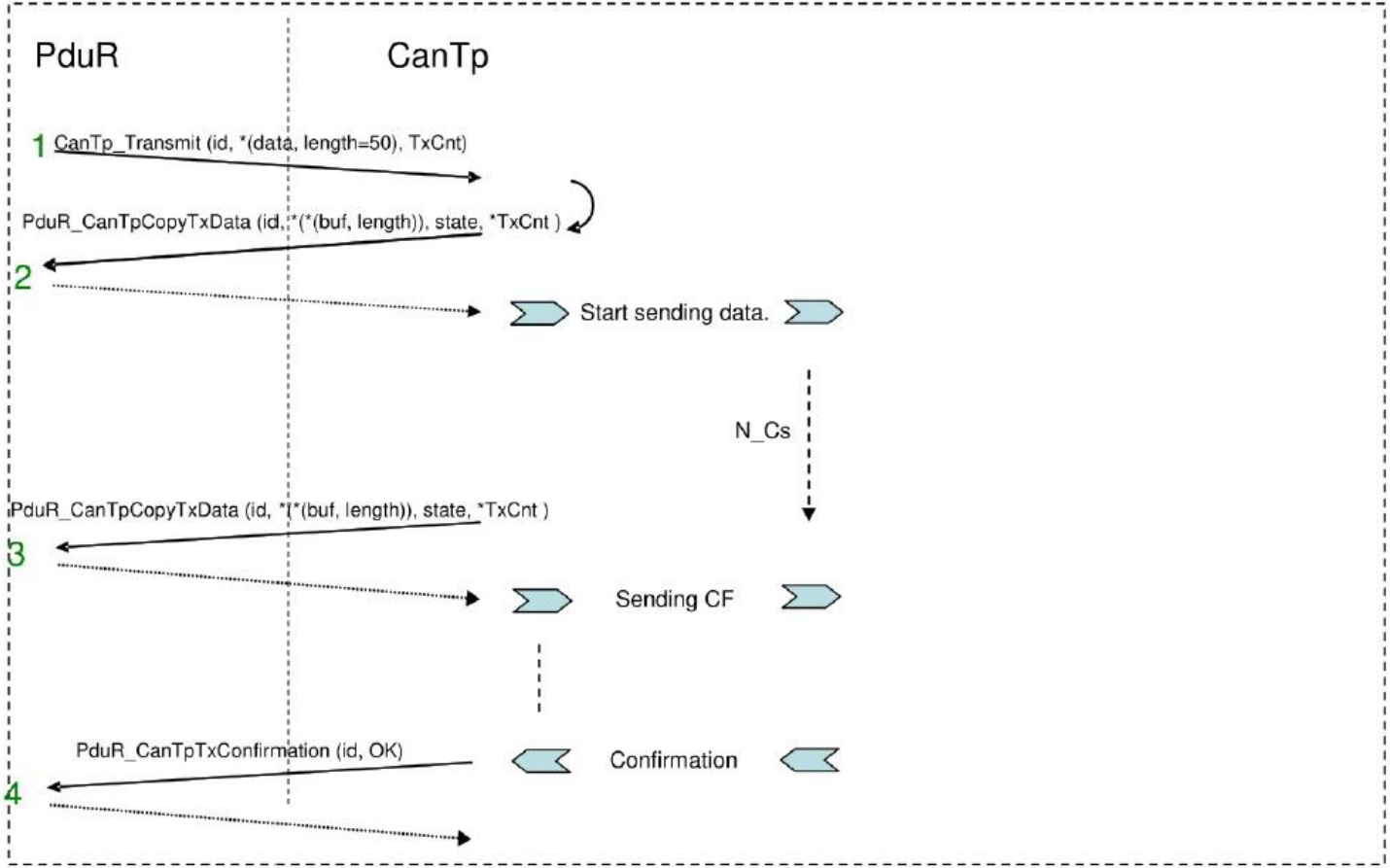



**Figure 7.4: Example of transmit process**

# Buffer strategy

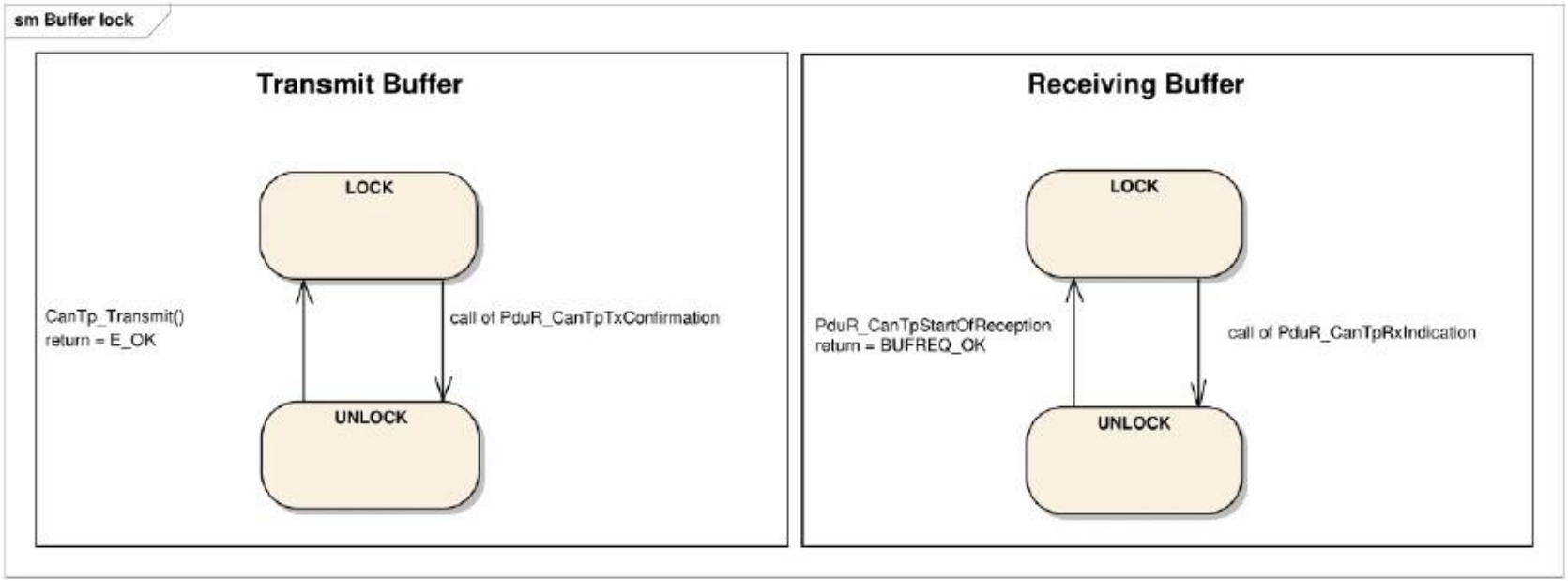



Figure 7.3: Tx and Rx Buffer locking

# Relationship between CAN NSduId and CAN LSduId

- A CAN NSduId shall only be linked to one CAN LSduId that is used to transmit SF, FF, FC and CF frames.

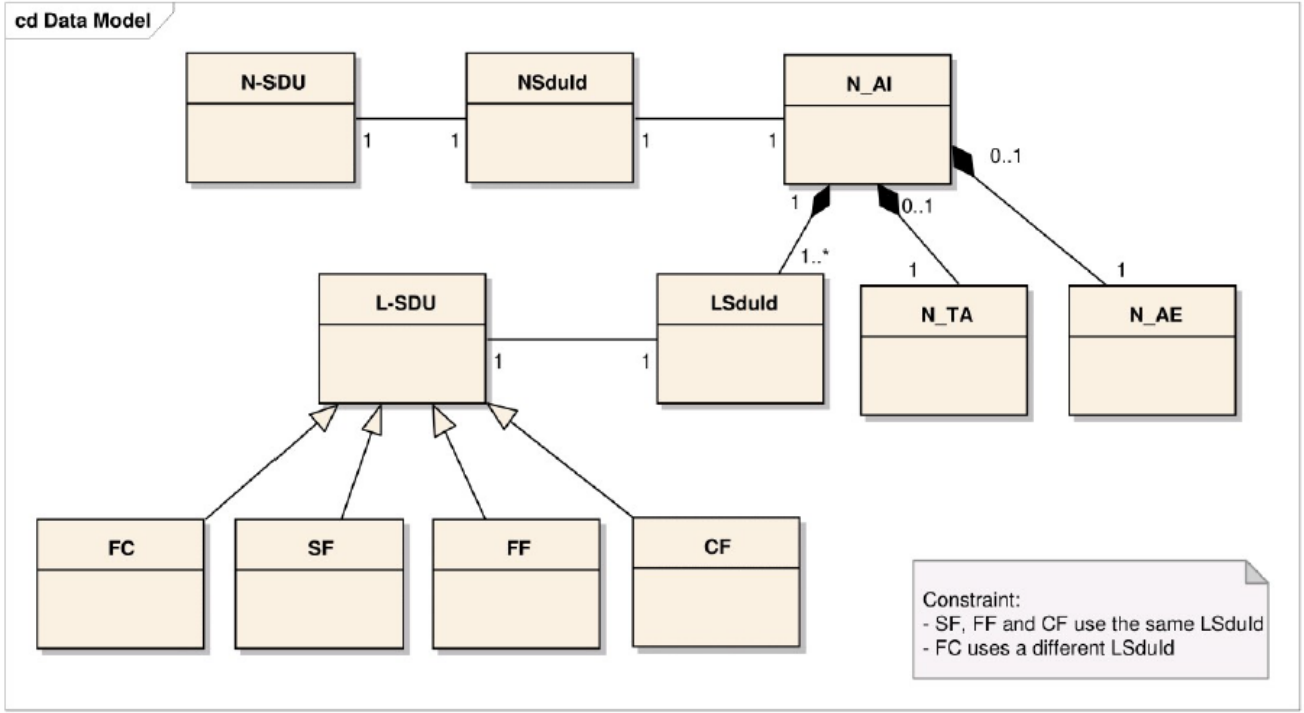



Figure 7.8: Possible links between NSduId and LSduId

# Concurrent connection

- The CAN Transport Layer is able to manage several connections simultaneously (e.g. a UDS and an OBD request can be received at the same time).
- The CanTp module shall support several connections simultaneously.
- All the necessary information (Channel number, Timing parameter..) is configured inside the CAN Transport Layer module.
- Each connection channel is independent of the other connection channels. This means that a connection channel uses its own resources, such as internal buffer, timer, or state machine.

# N-PDU padding

- To guarantee complete compatibility with all upper layer requirements concerning the frame data length (e.g. OBD requires data length to always be set to 8 bytes, however UDS does not), the padding activation is configurable at pre-compile time per N-SDU by using either CanTpRxPaddingActivation for a Rx N-SDU or CanTpTxPaddingActivation for a Tx N-SDU.

- The value used for padding bytes is configurable via configuration parameter CANTP_PADDING_BYTE

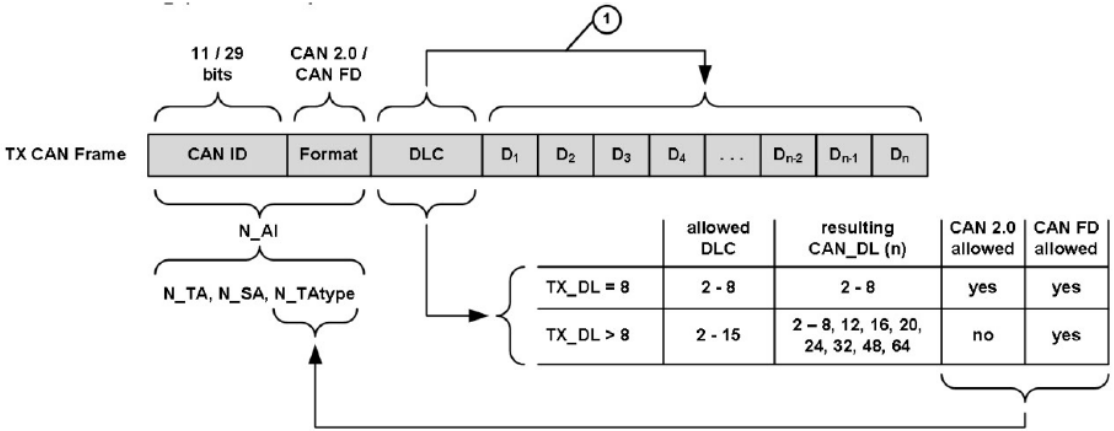



Figure 7.9: ISO frame construction

# Development Errors

| *Type of error* | *Related error code* | *Error value* |
|---|---|---|
| API service called with wrong parameter(s): When CanTp_ChangeParameter is called with invalid value. | CANTP_E_PARAM_CONFIG | 0x01 |
| API service called with wrong parameter(s): When CanTp_ChangeParameter or CanTp_Read Parameter is called with invalid parameter ID. | CANTP_E_PARAM_ID | 0x02 |
| API service called with a NULL pointer. | CANTP_E_PARAM_POINTER | 0x03 |
| Module initialization has failed, e.g. CanTp_Init() called with an invalid pointer in postbuild. | CANTP_E_INIT_FAILED | 0x04 |
| API service used without module initialization : On any API call except CanTp_Init(), CanTp_Get VersionInfo() and CanTp_MainFunction() if CanTp is in state CANTP_OFF | CANTP_E_UNINIT | 0x20 |
| Invalid Transmit PDU identifier (e.g. a service is called with an inexistent Tx PDU identifier) | CANTP_E_INVALID_TX_ID | 0x30 |
| Invalid Receive PDU identifier (e.g. a service is called with an inexistent Rx PDU identifier) | CANTP_E_INVALID_RX_ID | 0x40 |

# Runtime Errors

| *Type of error* | *Related error code* | *Error value* |
|---|---|---|
| PDU received with a length smaller than 8 bytes. (i.e. PduInfoPtr.SduLength < 8) | CANTP_E_PADDING | 0x70 |
| CanTp_Transmit() is called for a configured Tx I-Pdu with functional addressing and the length parameter indicates, that the message can not be sent with a SF | CANTP_E_INVALID_TATYPE | 0x90 |
| Requested operation is not supported - a cancel transmission/reception request for an N-SDU that it is not on transmission/reception process | CANTP_E_OPER_NOT_SUPPORTED | 0xA0 |
| Event reported in case of an implementation specific error other than a protocol timeout error during a reception or a transmission | CANTP_E_COM | 0xB0 |
| Event reported in case of a protocol timeout error during reception | CANTP_E_RX_COM | 0xC0 |
| Event reported in case of a protocol timeout error during transmission | CANTP_E_TX_COM | 0xD0 |

# Function definitions

| | |
|---|---|
| CanTp_Init | void CanTp_Init **(**const CanTp_ConfigType***** CfgPtr**)** |
| CanTp_GetVersionInfo | void CanTp_GetVersionInfo **(**Std_VersionInfoType***** versioninfo**)** |
| CanTp_Shutdown | void CanTp_Shutdown **(**void**)** |
| CanTp_Transmit | Std_ReturnType CanTp_Transmit **(**PduIdType TxPduId, const PduInfoType***** PduInfoPtr**)** |
| CanTp_CancelTransmit | Std_ReturnType CanTp_CancelTransmit **(**PduIdType TxPduId**)** |
| CanTp_CancelReceive | Std_ReturnType CanTp_CancelReceive **(**PduIdType RxPduId**)** |
| CanTp_ChangeParameter | Std_ReturnType CanTp_ChangeParameter **(**PduIdType id, TPParameterType parameter, uint16 value**)** |
| CanTp_ReadParameter | Std_ReturnType CanTp_ReadParameter **(**PduIdType id, TPParameterType parameter, uint16***** value**)** |
| CanTp_MainFunction | void CanTp_MainFunction **(**void **)** |

## Callback notifications

| | |
|---|---|
| CanTp_RxIndication | void CanTp_RxIndication **(** PduIdType RxPduId, const PduInfoType***** PduInfoPtr**)** |
| CanTp_TxConfirmation | void CanTp_TxConfirmation **(** PduIdType TxPduId, Std_ReturnType result **)** |

## Mandatory Interfaces

| | |
|---|---|
| CanIf_Transmit | CanIf.h |
| Det_ReportRuntimeError | Det.h |
| PduR_CanTpCopyRxData | PduR_CanTp.h |
| PduR_CanTpCopyTxData | PduR_CanTp.h |
| PduR_CanTpRxIndication | PduR_CanTp.h |
| PduR_CanTpStartOfReception | PduR_CanTp.h |
| PduR_CanTpTxConfirmation | PduR_CanTp.h |

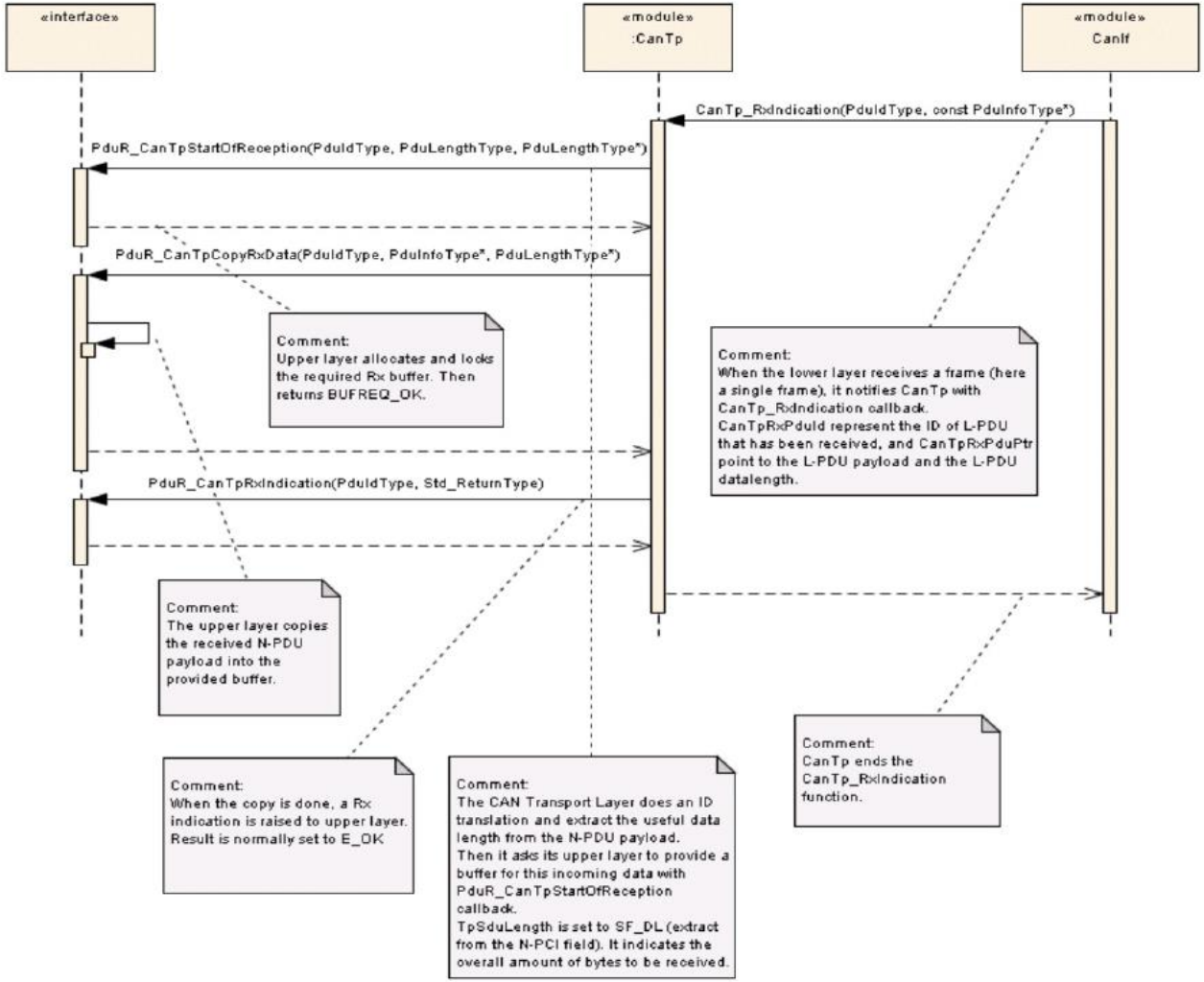



**Figure 9.2: Successful SF N-PDU reception**

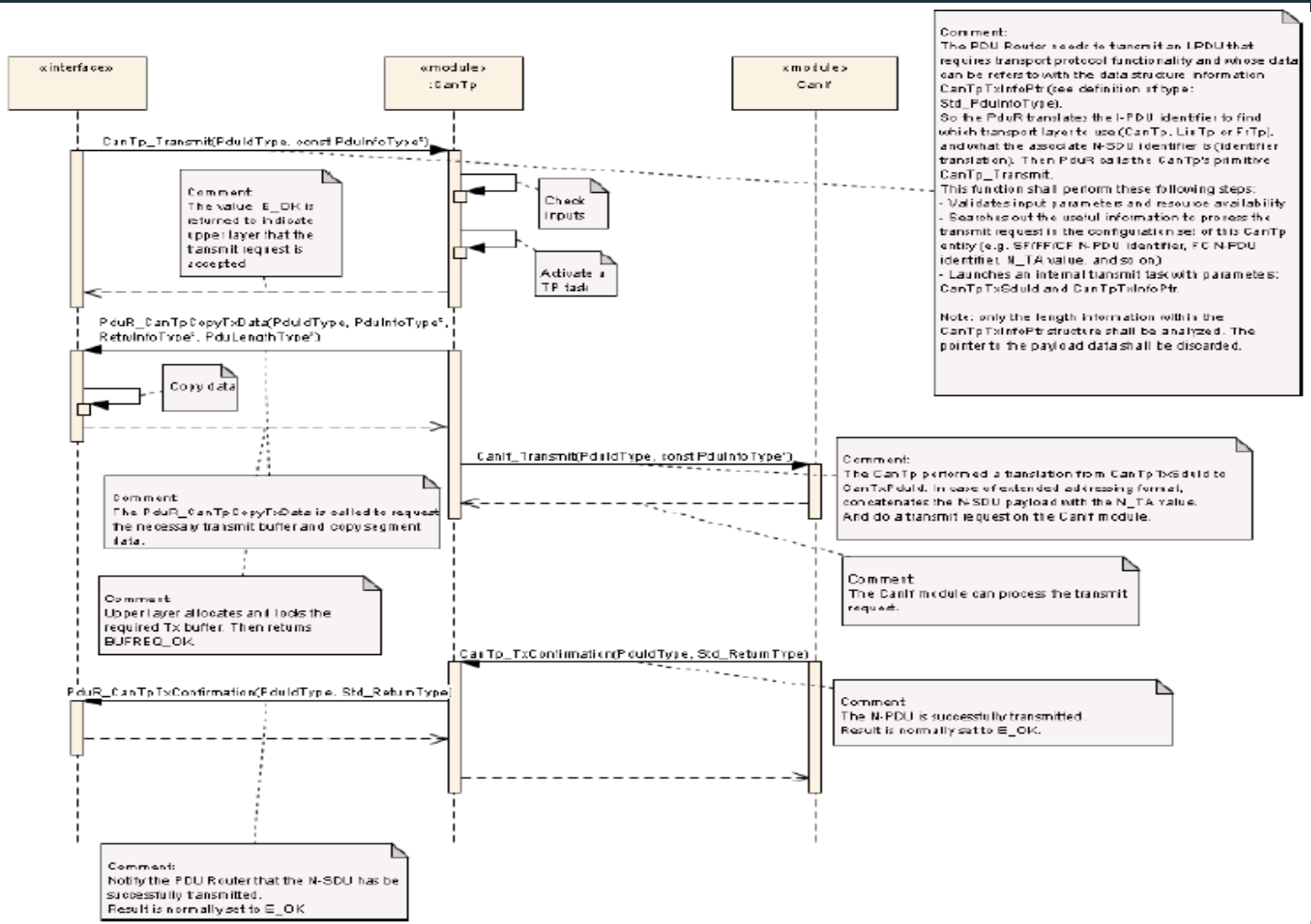



**Figure 9.3: Transmit request of SF N-SDU**

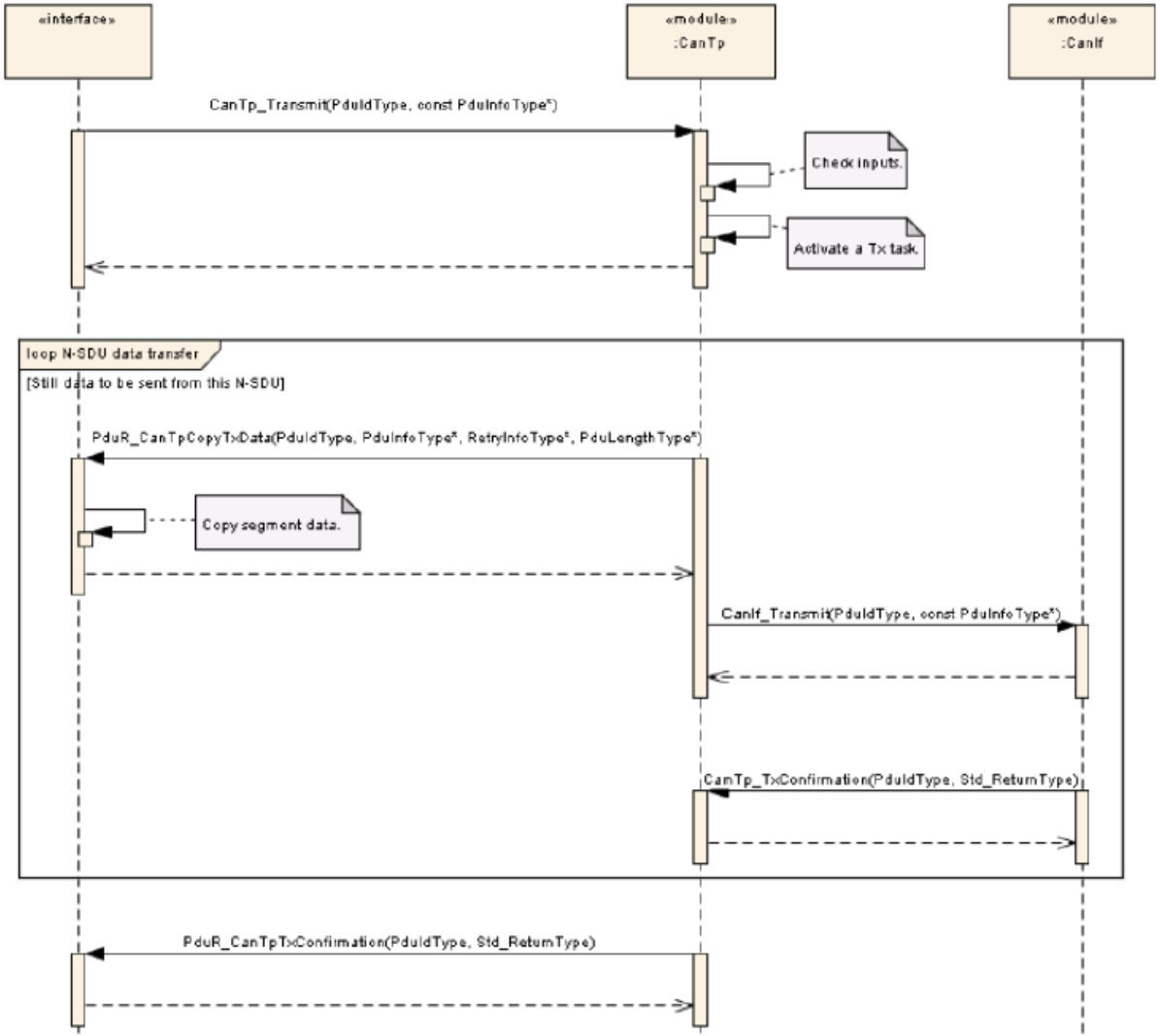



**Figure 9.4: Transmit request of larger N-SDU**

# Configuration specification

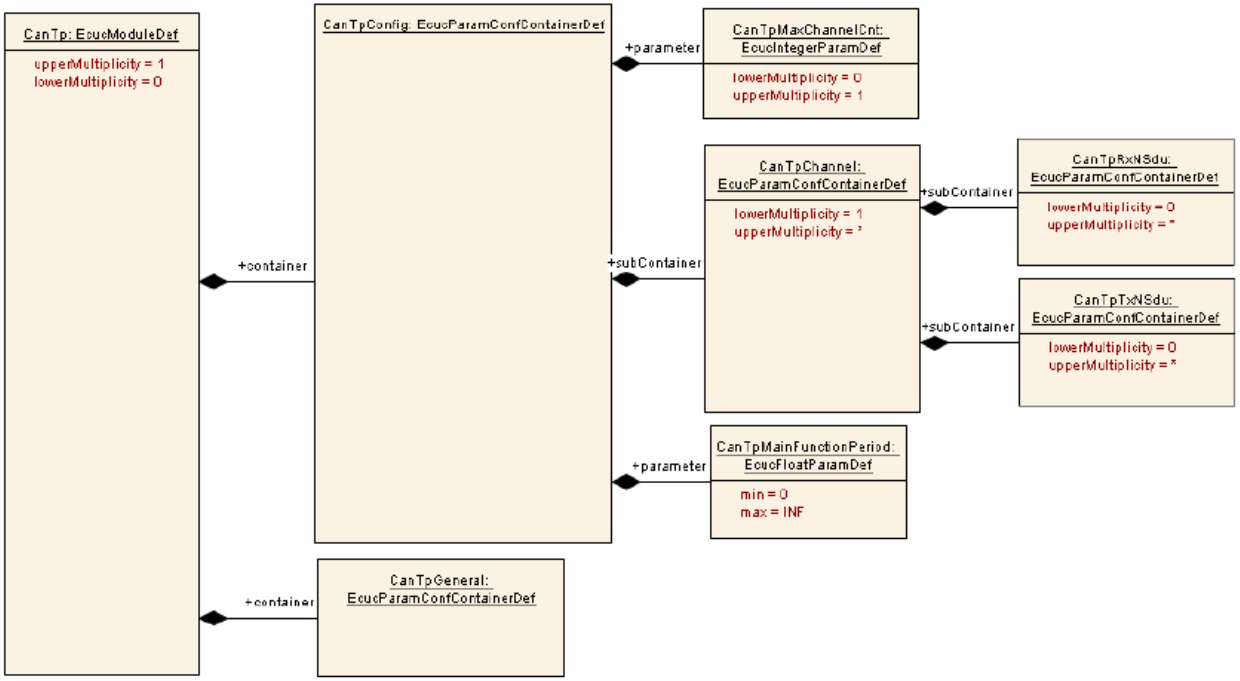



Figure 10.1: Configuration overview

# CanTpGeneral

| | |
|---|---|
| CanTpChangeParameterApi | This parameter, if set to true, enables the CanTp_ChangeParameterRequest Api for this Module. |
| CanTpDevErrorDetect | Switches the development error detection and notification on or off. |
| CanTpDynIdSupport | Enable support for dynamic ID handling via N-PDU MetaData. |
| CanTpFlexibleDataRateSupport | Enable support for CAN FD frames. |
| CanTpGenericConnectionSupport | Enable support for the handling of generic connections using N-SDUs with MetaData. Requires CanTpDynIdSupport. |
| CanTpPaddingByte | Used for the initialization of unused bytes with a certain value |
| CanTpReadParameterApi | if set to true, enables the CanTp_ReadParameterApi for this module. |
| CanTpVersionInfoApi | function CanTp_GetVersionInfo is configurable (On/Off) |

## CAN Tp Config

| | |
|---|---|
| CanTpMainFunctionPeriod | Configure the time for the MainFunction |
| CanTpMaxChannelCnt | Maximum number of channels. |
| CanTpChannel | Contains the configuration parameters of the CanTp channel. |

## CanTpChannel

| | |
|---|---|
| CanTpRxNSdu | The following parameters needs to be configured for each CAN N-SDU that the CanTp module receives via the CanTpChannel. |
| CanTpTxNSdu | The following parameters needs to be configured for each CAN N-SDU that the CanTp module transmits via the CanTpChannel. |

# CanTpRxNSdu

| | |
|---|---|
| CanTpBs | Sets the number of N-PDUs the CanTp receiver allows the sender to send, before waiting for an authorization to continue transmission of the following N-PDUs. |
| CanTpNar | Value in seconds of the N_Ar timeout. N_Ar is the time for transmission of a CAN frame (any N_PDU) on the receiver side. |
| CanTpNbr | Value in seconds of the performance requirement for (N_Br + N_Ar). N_Br is the elapsed time between the receiving indication of a FF or CF or the transmit confirmation of a FC, until the transmit request of the next FC. |
| CanTpNcr | Value in seconds of the N_Cr timeout. N_Cr is the time until reception of the next Consecutive Frame N_PDU. |
| CanTpRxAddressingFormat | CANTP_EXTENDED/ CANTP_MIXED/ CANTP_MIXED29BIT / CANTP_NORMALFIXED/ CANTP_STANDARD |
| CanTpRxNSduId | Unique identifier user by the upper layer to call CanTp_CancelReceive, CanTp_ChangeParameter and CanTp_ReadParameter. |
| CanTpRxPaddingActivation | Defines if the receive frame uses padding or not. This parameter is restricted to 8 byte N-PDUs. |
| CanTpRxTaType | CANTP_FUNCTIONAL / CANTP_PHYSICAL |
| CanTpRxWftMax | This parameter indicates how many Flow Control wait N-PDUs can be consecutively transmitted by the receiver. |
| CanTpSTmin | Sets the duration of the minimum time the CanTp sender shall wait between the transmissions of two CF NPDUs. |
| CanTpRxNSduRef | Reference to a Pdu in the COM-Stack. |

| | |
|---|---|
| CanTpNAe | This container is required for each RxNSdu and TxNSdu with AddressingFormat CANTP_MIXED or CANTP_MIXED29BIT. |
| CanTpNSa | This container is required for each RxNSdu and TxNSdu with RxTaType CANTP_PHYSICAL and CanTpAddressingFormat CANTP_EXTENDED. |
| CanTpNTa | This container is required for each RxNSdu and TxNSdu with AddressingFormat CANTP_EXTENDED. |
| CanTpRxNPdu | Used for grouping of the ID of a PDU and the Reference to a PDU.<br>This N-PDU consumes a meta data item of type CAN_ID_32. |
| CanTpTxFcNPdu | Used for grouping of the ID of a PDU and the Reference to a PDU.<br>This N-PDU produces a meta data item of type CAN_ID_32. |

## CanTpTxFcNPdu

| | |
|---|---|
| CanTpTxFcNPduConfirmationPduId | Handle Id to be used by the CanIf to confirm the transmission of the CanTpTxFcNPdu to the CanTP module. |
| CanTpTxFcNPduRef | Reference to a Pdu in the COM-Stack. |

## CanTpRxNPdu

| | |
|---|---|
| CanTpRxNPduId | The N-PDU identifier attached to the RxNsdu is identified by CanTpRxNSduId. |
| CanTpRxNPduRef | Reference to a Pdu in the COM-Stack. |

# CanTpTxNSdu

| | |
|---|---|
| CanTpNas | Value in second of the N_As timeout. N_As is the time for transmission of a CAN frame (any N_PDU) on the part of the sender. |
| CanTpNbs | Value in seconds of the N_Bs timeout. N_Bs is the time of transmission until reception of the next Flow Control N_PDU. |
| CanTpNcs | Value in seconds of the performance requirements relating to N_Cs. CanTpNcs is the time in which CanTp is allowed to request from PduR the Tx data of a Consecutive Frame N_PDU. |
| CanTpTc | Switch for enabling Transmit Cancellation. |
| CanTpTxAddressingFormat | CANTP_EXTENDED/ CANTP_MIXED/ CANTP_MIXED29BIT / CANTP_NORMALFIXED/ CANTP_STANDARD |
| CanTpTxNSduId | Unique identifier to a structure that contains all useful information to process the transmission of a TxNsdu. |
| CanTpTxPaddingActivation | Defines if the transmit frame use padding or not. This parameter is restricted to 8 byte N-PDUs. |
| CanTpTxTaType | CANTP_FUNCTIONAL / CANTP_PHYSICAL |
| CanTpTxNSduRef | Reference to a Pdu in the COM-Stack. |

| | |
|---|---|
| CanTpNAe | This container is required for each RxNSdu and TxNSdu with AddressingFormat CANTP_MIXED or CANTP_MIXED29BIT. |
| CanTpNSa | This container is required for each RxNSdu and TxNSdu with RxTaType CANTP_PHYSICAL and CanTpAddressingFormat CANTP_EXTENDED. |
| CanTpNTa | This container is required for each RxNSdu and TxNSdu with AddressingFormat CANTP_EXTENDED. |
| CanTpRxFcNPdu | Used for grouping of the ID of a PDU and the Reference to a PDU. This N-PDU consumes a meta data item of type CAN_ID_32. |
| CanTpTxNPdu | Used for grouping of the ID of a PDU and the Reference to a PDU. This N-PDU produces a meta data item of type CAN_ID_32. |

## CanTpRxFcNPdu

| | |
|---|---|
| CanTpRxFcNPduId | N-PDU identifier attached to the FC N-PDU of this TxNsdu identified by CanTpTxNSduId. |
| CanTpRxFcNPduRef | Reference to a Pdu in the COM-Stack. |

## CanTpTxNPdu

| | |
|---|---|
| CanTpTxNPduConfirmationPduId | Handle Id to be used by the CanIf to confirm the transmission of the CanTpTxNPdu to the CanIf module. |
| CanTpTxNPduRef | Reference to a Pdu in the COM-Stack. |

# Parent Container: CanTpRxNSdu, CanTpTxNSdu

| | |
|---|---|
| CanTpNTa | This parameter contains the transport protocol target address value. |
| CanTpNSa | This parameter contains the transport protocol source address value. |
| CanTpNAe | This container is required for each RxNSdu and TxNSdu with AddressingFormat CANTP_MIXED or CANTP_MIXED29BIT. |